第五章 神经网络

主要内容

5.1 神经网络简介

5.2 误差逆传播算法（BP）

5.3 其他常见神经网络模型

RBF、SOM、级联相关网络、Elman网络等

5.4 深度学习关联

5.1.1 什么是神经网络（学习）？

neural networks are massively parallel interconnected networks of simple (usually adaptive) elements and their hierarchical organizations which are intended to interact with the objects of the real world in the same way as biological nervous systems do.

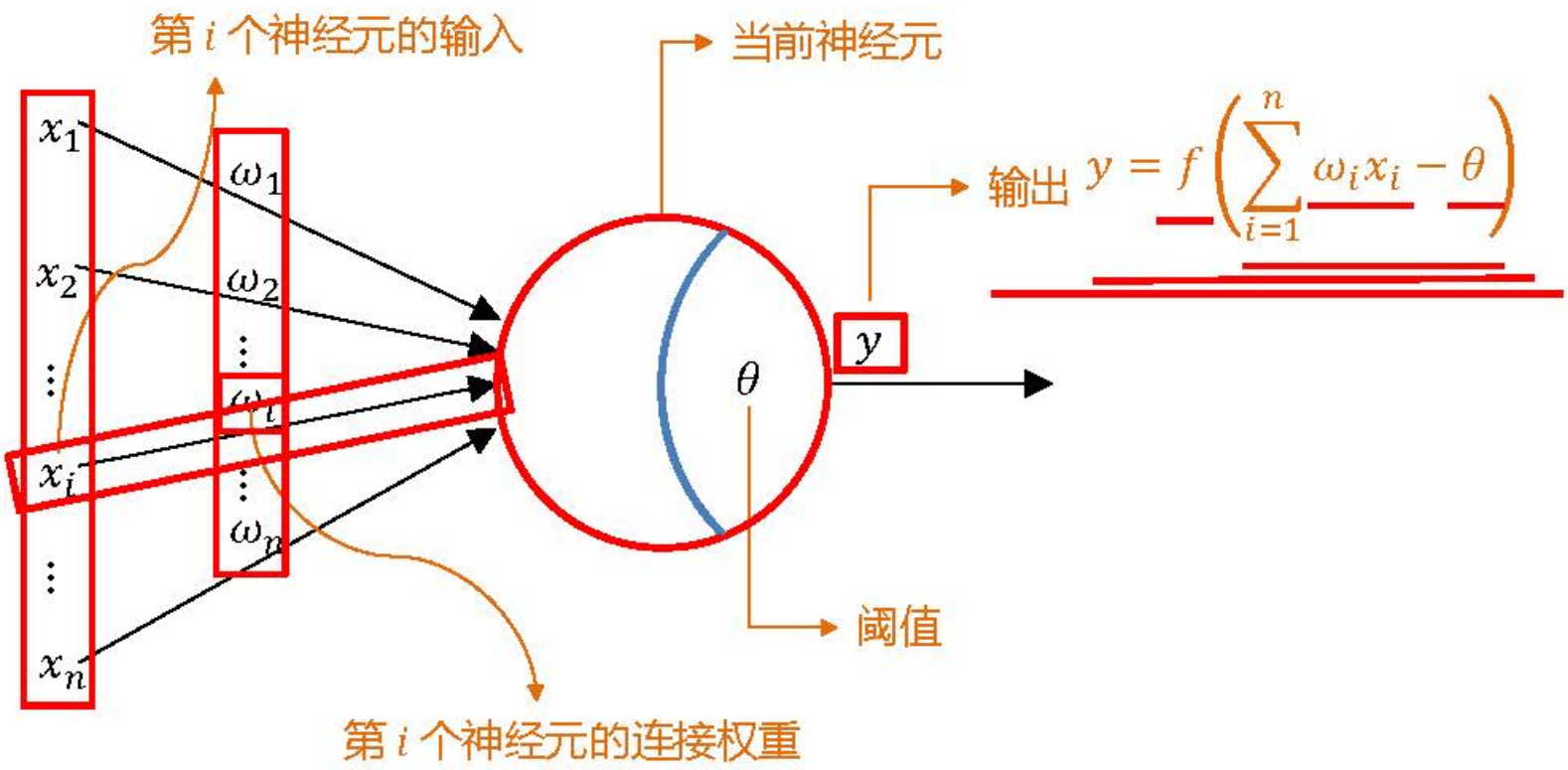
[T. Kohonen，1988，Neural Networks 创刊号]

- **神经网络**是一个具有**适应性**的**简单单元**组成的广泛并行互联的**网络**，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。

神经网络是一个很大的学科，本课程仅讨论它与机器学习的交集，“神经网络学习”亦称“连接主义(connectionism)”学习。

5.1.2 M-P神经元模型

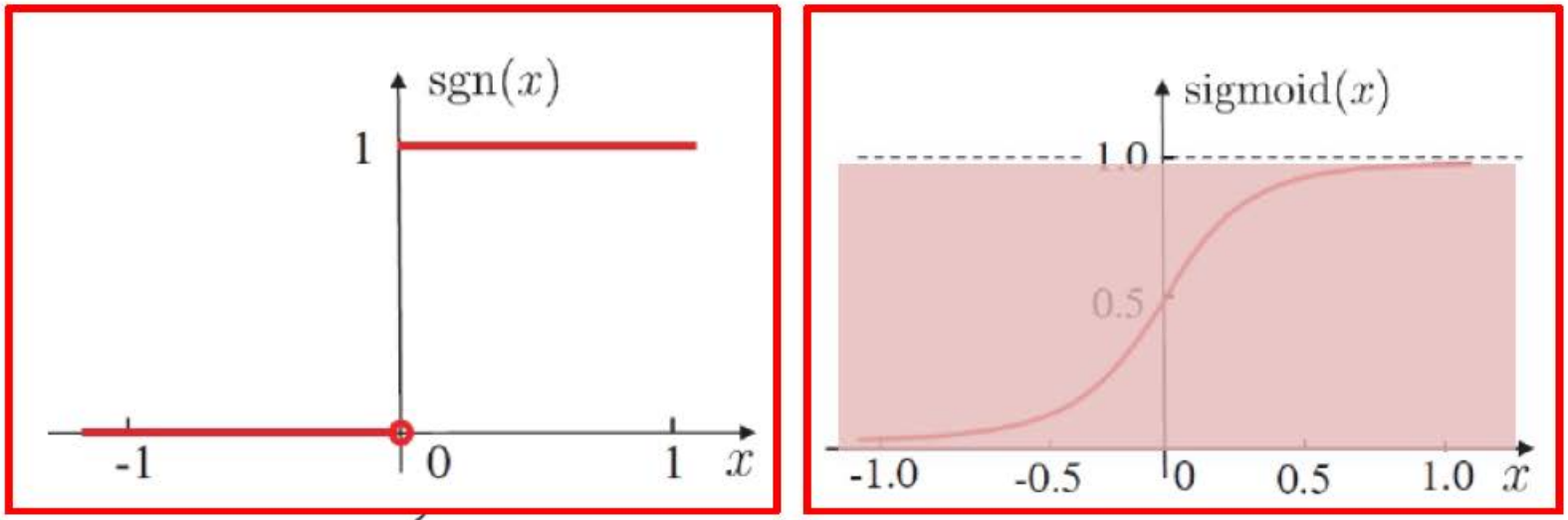
■ M-P 神经元模型 [McCulloch and Pitts, 1943]



□ 神经网络学得的知识蕴含在**连接权**与**阈值**中。

5.1.2 激活函数

- 理想激活函数是阶跃函数，0 表示抑制神经元而 1 表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质，常用的是 Sigmoid 函数



$$\mathrm{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数

$$\mathrm{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

(b) Sigmoid 函数

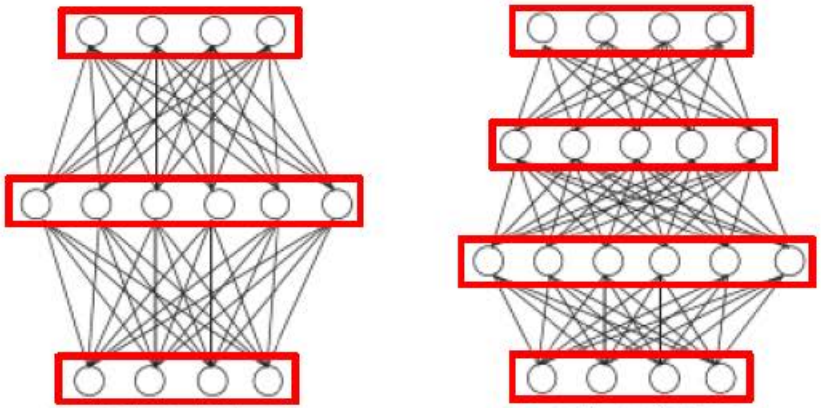
图 5.2 典型的神经元激活函数

5.1.3 多层前馈网络结构

多层网络：包含隐层的网络

前馈网络：神经元之间不存在同层连接也不存在跨层连接，即网络中无环或者回路。

隐层和输出层神经元亦称“功能单元”（functional unit），无隐藏层的又称“感知机(Perceptron)



(a) 单隐层前馈网络　　(b) 双隐层前馈网络

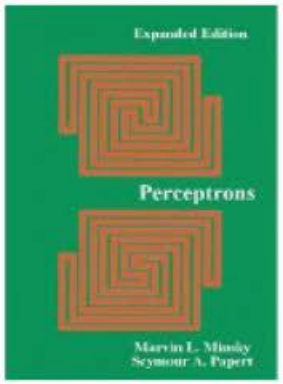
- 多层前馈网络有强大的表示能力

> 只需一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数
>
> [Hornik et al., 1989]

- 如何设置隐层神经元数是**未决**问题．实际常用“试错法”！

5.1.4 神经网络发展回顾

- 1940年代—萌芽期：M-P模型（1943），Hebb 学习规则（1945）
- 1958左右—1969左右 ~繁荣期：感知机（1958），Adaline (1960)，…
- 1969年：Minsky & Papert “Perceptrons”



冰河期

- 1985左右 -1995左右~繁荣期：Hopfield (1983)，BP (1986)，…
- 1995年左右：SVM 及 统计学习 兴起

沉寂期

热十（年）
冷十五（年）

- 2010左右 -至今 ~繁荣期 ：深度学习

启示

科学的发展总是“螺旋式上升”

三十年河东、三十年河西

坚持才能有结果！

追热门、赶潮流 —— 三思而后行

主要内容

5.1 神经网络简介

5.2 误差逆传播算法（BP）

5.3 其他常见神经网络模型

RBF、SOM、级联相关网络、Elman网络等

5.4 深度学习关联

5.2 误差逆传播算法（BP）

最成功、最常用的神经网络算法，可被用于多种任务（不仅限于分类）

P. Werbos在博士学位论文中正式提出:

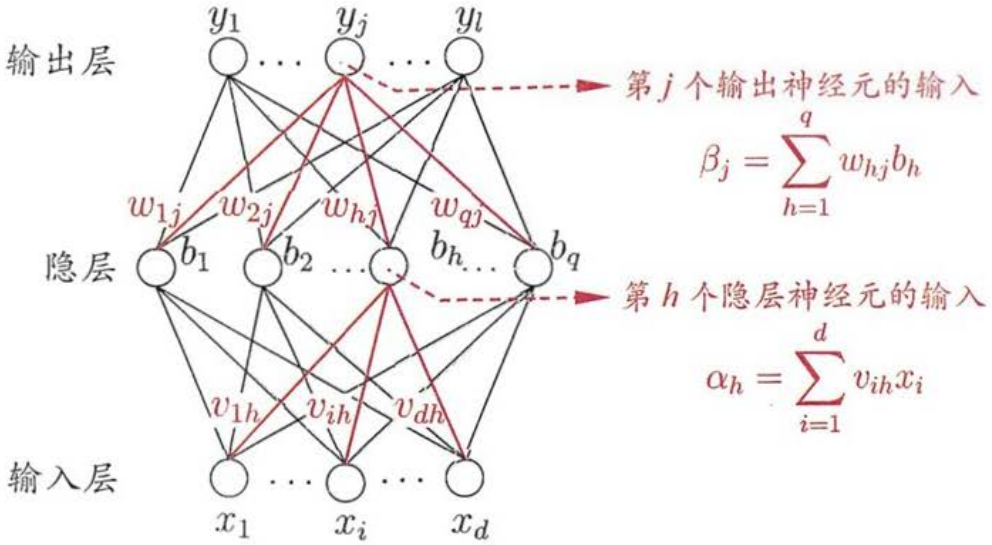
P. Werbos. Beyond regression: New tools for prediction and analysis in the behavioral science. Ph.D dissertation, Harvard University, 1974

给定训练集 $D = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \ldots, (\boldsymbol{x}_m, \boldsymbol{y}_m)\}$, $\boldsymbol{x}_i \in \mathbb{R}^d$, $\boldsymbol{y}_i \in \mathbb{R}^l$

输入： d 维特征向量
输出： l 个输出值

隐层：假定使用 q 个隐层神经元

假定功能单元均使用 Sigmoid 函数



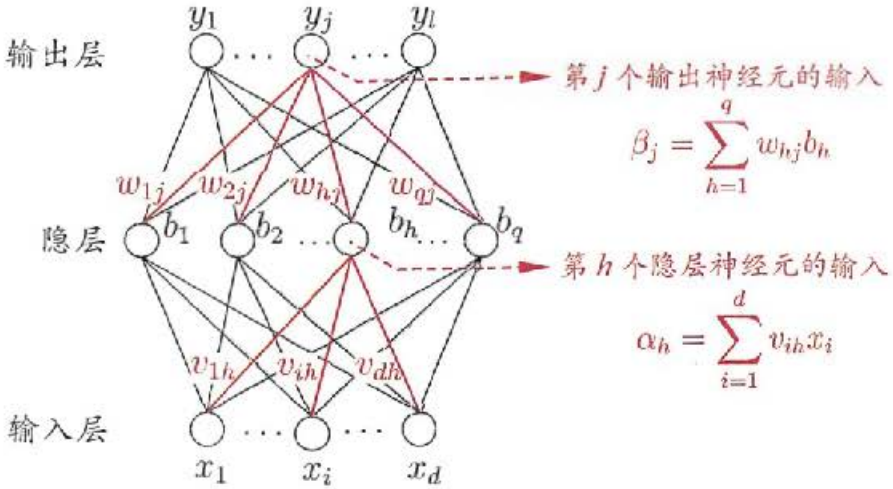
5.2.1 BP 算法推导

对于训练例$(\boldsymbol{x}_k, \boldsymbol{y}_k)$，假定网络的实际输出为 $\hat{\boldsymbol{y}}_k = (\hat{y}_1^k, \hat{y}_2^k, \ldots, \hat{y}_l^k)$

$$\hat{y}_j^k = f(\beta_j - \theta_j)$$

则网络在$(\boldsymbol{x}_k, \boldsymbol{y}_k)$上的均方误差为：

$$E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$



需通过学习确定的参数数目：$(d+l+1)q+l$

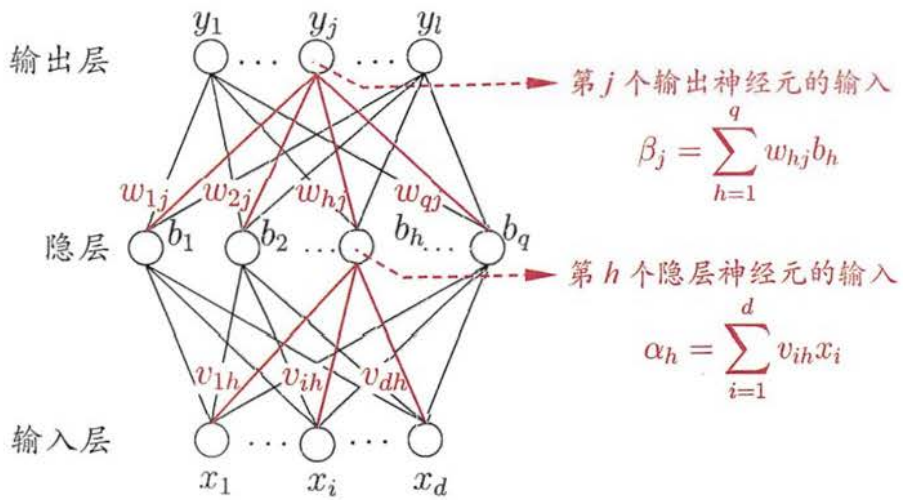
BP 是一个迭代学习算法，在迭代的每一轮中采用如下误差修正：

$$v \leftarrow v + \boxed{\triangle v}$$

5.2.1 BP 算法推导

BP 算法基于 梯度下降 策略，以目标的负梯度方向对参数进行调整

以 w_{hj} 为例



对误差 E_k，给定学习率 η，有：

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

注意到 w_{hj} 先影响到 β_j

再影响到 $\hat{y}_j^k$，然后才影响到 E_k，有：

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}}$$

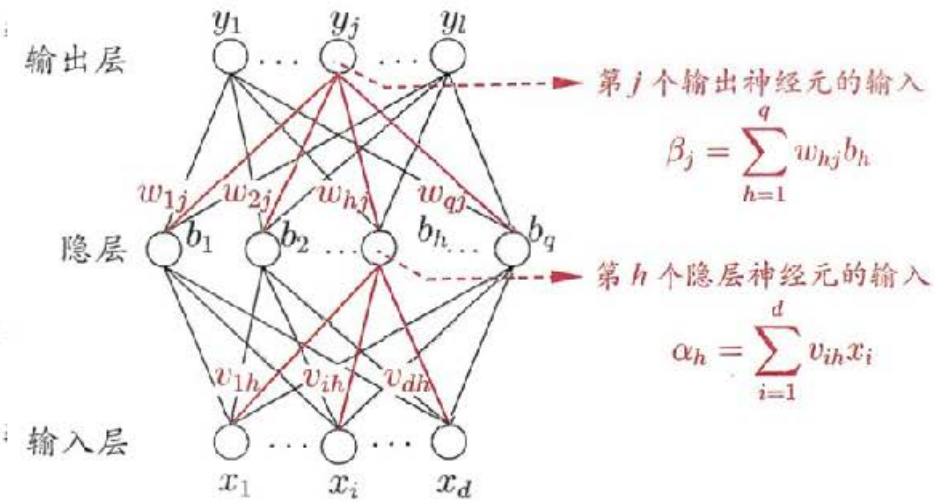
"链式法则"

5.2.1 BP 算法推导(续)

$$\hat{y}_j = f(\beta_j - \theta_j)$$

$$\frac{\partial E_k}{\partial w_{hj}} = \boxed{\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j}} \cdot \boxed{\frac{\partial \beta_j}{\partial w_{hj}}}$$

$$= b_h$$



$$g_j = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j}$$

$$= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j)$$

$$= \underline{\hat{y}_j^k (1 - \hat{y}_j^k)} (y_j^k - \hat{y}_j^k) .$$

于是，

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} = \eta g_j b_h$$

对 $\mathrm{sigmoid}(x) = \dfrac{1}{1+e^{-x}}$, 有

$$f'(x) = f(x)(1 - f(x))$$

再注意到 $\hat{y}_j^k = f(\beta_j - \theta_j)$

5.2.1 BP 算法推导(续)

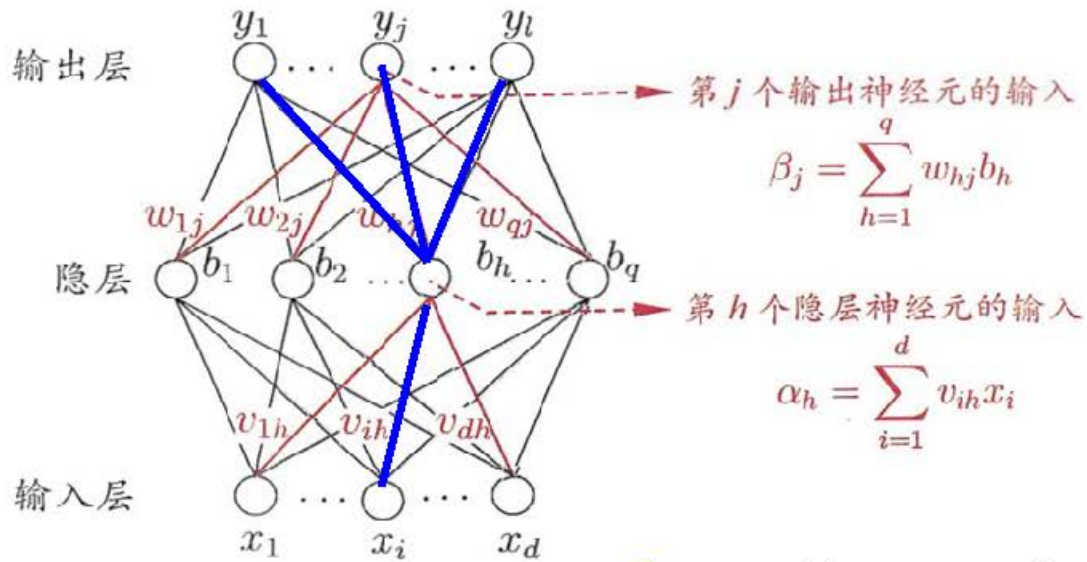
$$\hat{y}_j = f(\beta_j - \theta_j)$$

类似地，有：

$$\Delta\theta_j = -\eta g_j$$

$$\Delta v_{ih} = \eta e_h x_i$$

$$\Delta\gamma_h = -\eta e_h$$



$$b_h = f(\alpha_h - \gamma_h)$$

其中：

$$e_h = -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h}$$

$$= -\sum_{j=1}^{l} \frac{\partial E_k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} f'(\alpha_h - \gamma_h) = \sum_{j=1}^{l} w_{hj} g_j f'(\alpha_h - \gamma_h)$$

$$= b_h(1 - b_h) \sum_{j=1}^{l} w_{hj} g_j$$

$\eta \in (0, 1)$ 不能太大、不能太小

5.2.2 BP 算法

预处理：属性值一般伸缩到[-1,1], Y伸缩到[0,1]

输入： 训练集 $D = \{(\boldsymbol{x}_k, \boldsymbol{y}_k)\}_{k=1}^{m}$;
学习率 η.

过程：

1: 在$(0,1)$范围内随机初始化网络中所有连接权和阈值
2: **repeat**
3: **for all** $(\boldsymbol{x}_k, \boldsymbol{y}_k) \in D$ **do**
4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\boldsymbol{y}}_k$;
5: 根据式(5.10) 计算输出层神经元的梯度项 g_j;
6: 根据式(5.15) 计算隐层神经元的梯度项 e_h;
7: 根据式(5.11)-(5.14) 更新连接权 w_{hj}, v_{ih} 与阈值 θ_j, γ_h
8: **end for**
9: **until** 达到停止条件

- 设置最大迭代次数
- 计算训练集在网络上的预测准确率

输出： 连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

标准BP算法 vs. 累积BP算法

$$E_k = \sum_{j=1}^{l}\left(\hat{y}_j^{(k)} - y_j^{(k)}\right)^2$$

$$E = \frac{1}{m}\sum_{k=1}^{m} E_k = \frac{1}{2m}\sum_{k=1}^{m}\sum_{j=1}^{l}\left(\hat{y}_j^{(k)} - y_j^{(k)}\right)^2$$

标准 BP 算法

- 每次针对单个训练样例更新权值与阈值
- 参数更新频繁，不同样例可能抵消，需要**多次迭代**

累积 BP 算法

- 其优化目标是最小化整个训练集上的累计误差
- 读取整个训练集一遍才对参数进行更新，参数更新频率**较低**

在很多任务中，累计误差下降到一定程度后，进一步下降会非常缓慢，这时标准BP算法往往会获得较好的解，尤其当训练集非常大时效果更明显。

5.2.3 缓解过拟合

BP算法常常导致过拟合

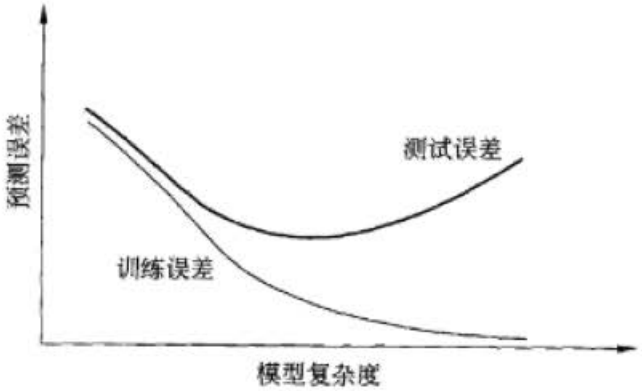


图 1.3 训练误差和测试误差与模型复杂度的关系

主要策略：

- 早停 (early stopping)
  - 若训练误差连续 a 轮的变化小于 b, 则停止训练
  - 使用验证集：若训练误差降低、验证误差升高 , 则停止训练

- 正则化 (regularization)
  - 在误差目标函数中增加一项描述网络复杂度

例如 $$E = \lambda \frac{1}{m} \sum_{k=1}^{m} E_k + \underline{(1-\lambda) \sum_i w_i^2}$$

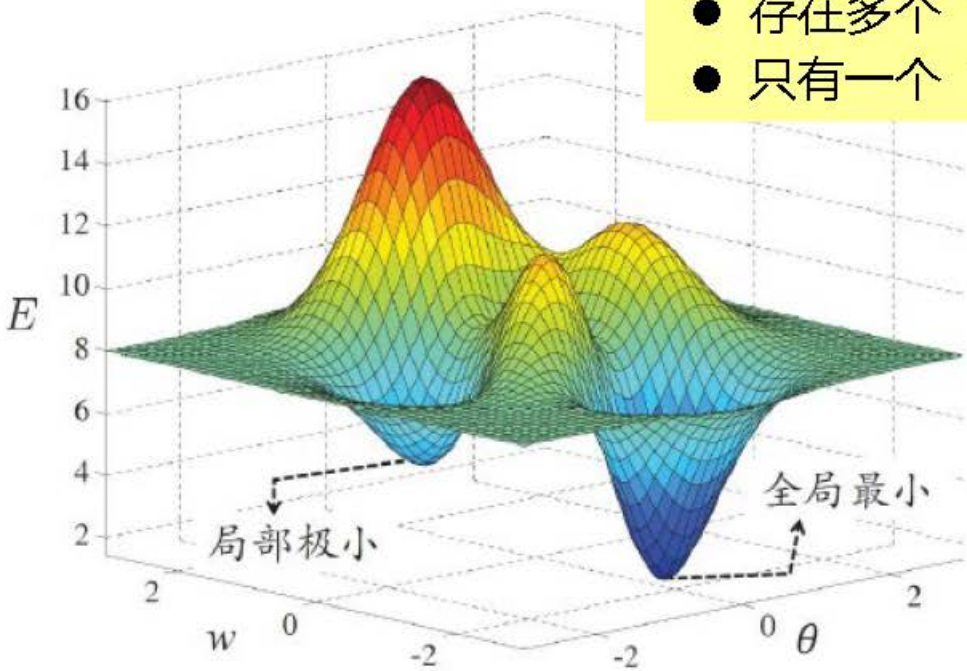
偏好比较小的连接权和阈值，使网络输出更“光滑”

全局最小 vs. 局部极小

神经网络的训练过程可看作一个参数寻优过程：

在参数空间中，寻找一组最优参数使得误差最小

- 存在多个“局部极小”
- 只有一个“全局最小”



“跳出”局部极小的常见策略：

- ✓ 不同的初始参数
- ✓ 模拟退火
- ✓ 随机扰动
- ✓ 遗传算法

……

全局最小 vs. 局部极小

- 以多组不同参数值初始化多个神经网络，按标准方法训练后，取其中误差最小的解作为最终参数。这相当于从多个不同的初始点开始搜索，这样就可能陷入不同的局部极小，从中进行选择有可能获得更接近全局最小的结果。
- 使用“模拟退火”(simulated annealing)技术[Aarts and Korst，1989]模拟退火在每一步都以一定的概率接受比当前解更差的结果，从而有助于“跳出”局部极小。在每步迭代过程中，接受“次优解”的概率要随着时间的推移而逐渐降低，从而保证算法稳定。
- 使用随机梯度下降。与标准梯度下降法精确计算梯度不同，随机梯度下降法在计算梯度时加入了随机因素。于是，即便陷入局部极小点，它计算出的梯度仍可能不为零，这样就有机会跳出局部极小继续搜索。
- 遗传算法(genetic algorithms)[Goldberg，1989] 也常用来训练神经网络以更好地逼近全局最小。需注意的是，上述用于跳出局部极小的技术大多是启发式，理论上尚缺乏保障。

主要内容

5.1 神经网络简介

5.2 误差逆传播算法（BP）

5.3 其他常见神经网络模型

RBF、SOM、级联相关网络、Elman网络等

5.4 深度学习关联

5.3 其他常见神经网络模型

- **RBF：** 分类任务中除BP之外最常用
- ART：“竞争学习”的代表
- **SOM：**最常用的聚类方法之一
- **级联相关网络：**“构造性”神经网络的代表
- **Elman网络：**递归神经网络的代表
- Boltzmann机：“基于能量的模型”的代表
- ……

5.3.1 RBF 神经网络

RBF: Radial Basis Function (径向基函数)

其中 q 为隐层神经元个数, $\boldsymbol{c}_i$ 和 w_i 分别是第 i 个隐层神经元所对应的中心和权重, $\rho(\boldsymbol{x}, \boldsymbol{c}_i)$ 是径向基函数, 这是某种沿径向对称的标量函数, 通常定义为样本 $\boldsymbol{x}$ 到数据中心 $\boldsymbol{c}_i$ 之间欧氏距离的单调函数. 常用的高斯径向基函数形如

例如高斯径向基函数 $\rho(\boldsymbol{x}, \boldsymbol{c}_i) = e^{-\beta_i \|\boldsymbol{x}-\boldsymbol{c}_i\|^2}$

- 输出层是隐层神经元输出的线性组合

$$\varphi(\boldsymbol{x}) = \sum_{i=1}^{q} w_i \rho(\boldsymbol{x}, \boldsymbol{c}_i)$$

训练:

Step1：确定神经元中心，常用的方式包括随机采样、聚类等

Step2：利用BP算法等确定参数

SOM 的训练过程很简单：在接收到一个训练样本后，每个输出层神经元会计算该样本与自身携带的权向量之间的距离，距离最近的神经元成为竞争获胜者，称为最佳匹配单元(best matching unit)．然后，最佳匹配单元及其邻近神经元的权向量将被调整，以使得这些权向量与当前输入样本的距离缩小．这个过程不断迭代，直至收敛．

- 竞争型的无监督神经网络
- 将高维数据映射到低维空间（通常为 2 维），高维空间中相似的样本点映射到网络输出层中邻近神经元
- 每个神经元拥有一个权向量
- 目标：为每个输出层神经元找到合适的权向量以保持拓扑结构

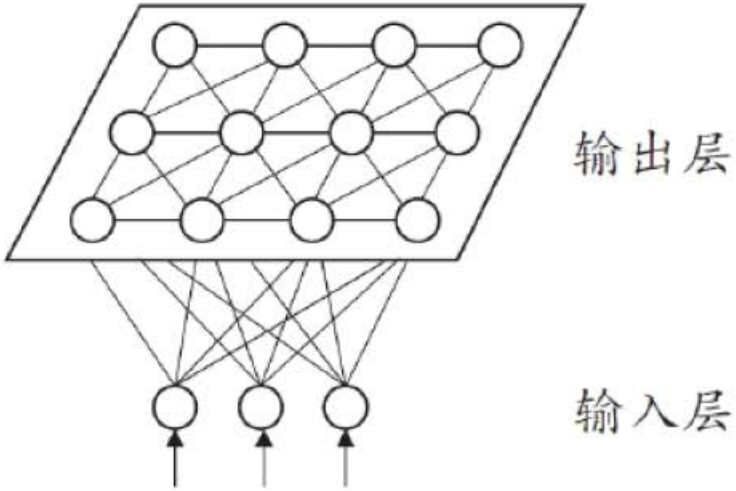


图 5.11　SOM 网络结构

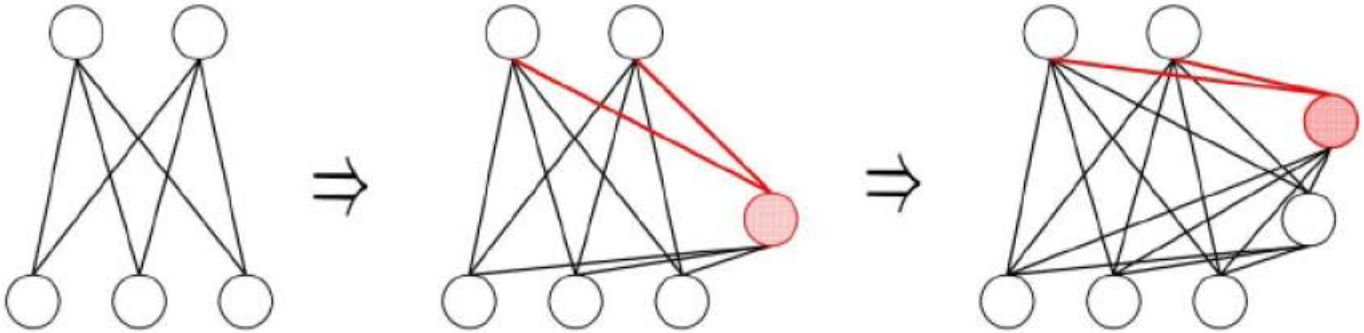
如图 5.11 所示，SOM 网络中的输出层神经元以矩阵方式排列在二维空间中，每个神经元都拥有一个权向量，网络在接收输入向量后，将会确定输出层获胜神经元，它决定了该输入向量在低维空间中的位置．SOM 的训练目标就是为每个输出层神经元找到合适的权向量，以达到保持拓扑结构的目的．

一般的神经网络模型通常假定网络结构是事先固定的，训练的目的是利用训练样本来确定合适的连接权、阈值等参数．与此不同，结构自适应网络则将网络结构也当作学习的目标之一，并希望能在训练过程中找到最符合数据特点的网络结构．级联相关(Cascade-Correlation)网络 [Fahlman and Lebiere, 1990] 是结构自适应网络的重要代表．

构造性神经网络： 将网络的结构也当做学习的目标之一， 希望在训练过程中找到适合数据的网络结构

训练：

- 开始时只有输入层和输出层
- 级联 - 新的隐层结点逐渐加入，从而创建起层级结构
- 相关 - 最大化新结点的输出与网络误差之间的相关性



(a) 初始状态　(b) 增加一个隐层结点　(c) 增加第二个隐层结点

5.3.4 Elman 网络

递归神经网络： Recurrent NN, 亦称 Recursive NN

- 网络中可以有环形结构, 可让使一些神经元的输出反馈回来作为输入
- t 时刻网络的输出状态：由 t 时刻的输入状态和 t-1 时刻的网络状态共同决定

Elman 网络是最常用的递归神经网络之一

- 结构与前馈神经网络很相似，但隐层神经元的输出被反馈回来
- 使用推广的BP算法训练

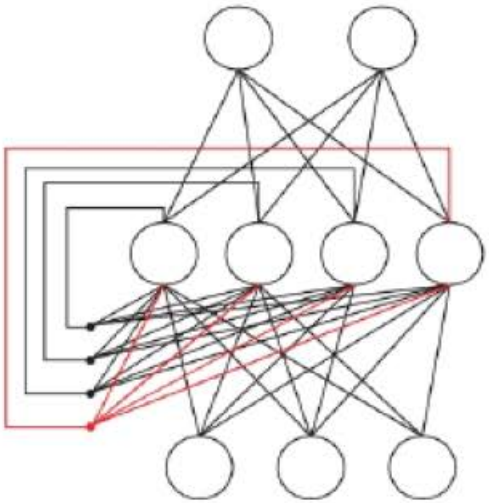


图 5.13 Elman 网络结构

目前在自然语言处理等领域常用的 LSTM 网络，

是一种复杂得多的递归神经网络

主要内容

5.1 神经网络简介

5.2 误差逆传播算法（BP）

5.3 其他常见神经网络模型

RBF、SOM、级联相关网络、Elman网络等

5.4 深度学习关联

5.4.1 深度学习的兴起

- 1943年，McCulloch和Pitts提出了**人工神经元**模型，**奠定了神经网络的理论基础。**
- 1957年，Rosenblatt提出了**感知机模型**，该模型被认为是神经网络的第一个实际应用。
- 1986年，Rumelhart等人提出了**反向传播算法**，使得神经网络的训练和应用更加实用。
- 1990年代初期，深度学习的研究进入了**低谷期**，主要原因是**缺乏数据和计算能力**。
- 2006年，Hinton等人提出了**深度信念网络模型**，为深度学习的研究和应用提供了新的思路。发表了深度学习的 Nature 文章。
- 2010年，Hinton等人提出了深度学习在语音识别任务中的应用，这是深度学习在实际应用中取得的首个突破。
- 2012年，Krizhevsky等人提出了**深度卷积神经网络（CNN）**在图像识别任务中的应用，该模型在**ImageNet数据集**上取得了优异的成绩，引起了学术界和工业界的广泛关注。

ImageNet竞赛：以超过第二名 10个百分点的成绩夺得当年竞赛的冠军

5.4.1 深度学习的兴起

Images & Video

Text & Language

- 2014年，Goodfellow等人提出了生成对抗网络（GAN
量样本的能力，为深度学习在图像、音频等领域的应
- 2015年，Google推出了TensorFlow深度学习框架，使
简单和高效。
- 2017年，OpenAI推出了**GPT模型**，该模型在自然语言
绩，引起了广泛的关注。

- 2023年，“**大模型元年**”。标志着自然语言处理（N

大语言模型爆发式：ChatGPT、GPT-4、 GPT-4 Turbo …

国内：ERNIE、XLNet、T5、… GPT-3（中文版）

智谱清言GLM-4、阿里巴巴Qwen-Max、百度文心一言4.0

5.4.1 深度学习的兴起

- 2024年，“**大模型落地的元年**”。侧重于中低风险的内部场景实施，人工智能等相关技术和产业链**进一步平民化**。
  - **GPT - 4o**（OpenAI）：拥有1.8万亿参数的混合专家（MoE）架构，支持多模态输入与实时交互。
  - **Gemini Ultra**（谷歌）：拥有1.56万亿参数，支持132种语言实时翻译，适用于智能制造和多语言会议实时转录。
  - **DeepSeek - R1/V3**（中国）：采用UltraMem稀疏架构，推理速度提升2-6倍，训练成本仅为GPT - 4o的1/70; **V3**（671B参数，MoE架构）
  - **Qwen2.5**（智谱AI）：130 亿个参数，并且是基于 GLM-4 架构进行微调
  - **Wan2.1**（阿里）：开源6天，Hugging Face&魔搭社区，模型热榜&模型空间榜

地区	代表模型	单次训练成本	硬件供应商	算力利用率
北美	GPT - 4o	2.3 亿 $	英伟达 DGX H100	92%
欧洲	Aleph Alpha	1.8 亿 €	自研量子计算芯片	85%
中国	DeepSeek - R1	340 万 $	英伟达 A100 集群	95%
韩国	HyperClova X	9500 万 $	三星 NPU 集群	78%

5.4.2 最常用的深度学习模型：卷积神经网络

CNN: Convolutional NN [LeCun and Bengio, 1995; LeCun et al., 1998]

每个卷积层包含多个特征映射，每个特征映射是一个由多个神经元构成的“平面”，通过一种卷积滤波器提取输入的一种特征

采样层亦称“汇合（pooling）层”，其作用是基于局部相关性原理进行亚采样，从而在减少数据量的同时保留有用信息

连接层就是传统神经网络对隐层与输出层的全连接

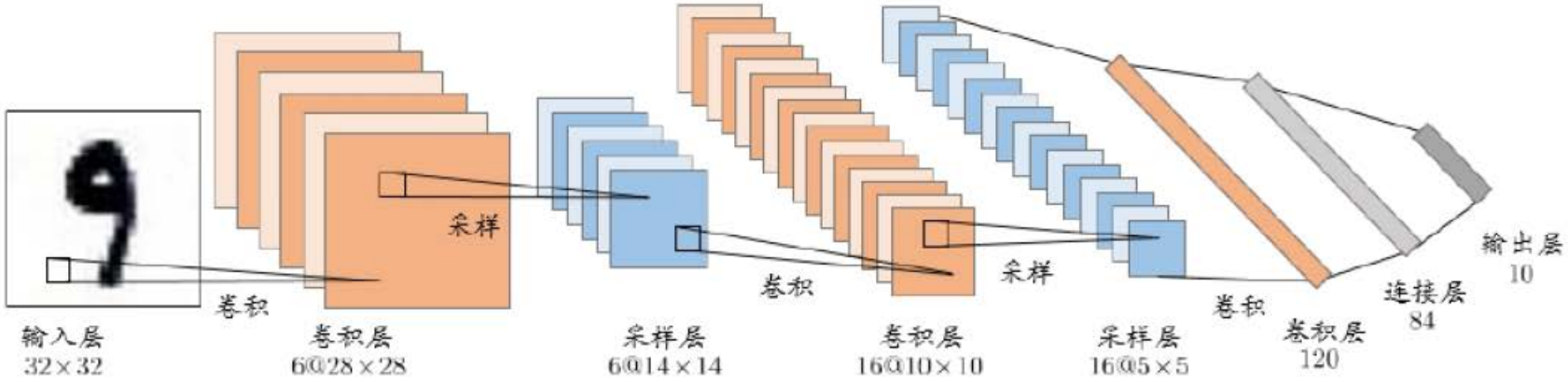


图 5.15 卷积神经网络用于手写数字识别 [LeCun et al., 1998]

5.4.3 深度学习

典型的深度学习模型就是很深层的神经网络

（例如，微软研究院2015年在ImageNet竞赛获胜使用152层网络）

提升模型复杂度 → 提升学习能力

- 增加隐层神经元数目（模型宽度）
- 增加隐层数目（模型深度）

增加隐层数目比增加隐层神经元数目更有效

不仅增加了拥有激活函数的神经元数，还增加了激活函数嵌套的层数

提升模型复杂度 → 增加过拟合风险；增加训练难度

- 过拟合风险：使用大量训练数据
- 训练困难：使用若干启发式诀窍

误差梯度在多隐层内传播时，往往会发散而不能收敛到稳定状态，因此，难以直接用经典BP算法训练

5.4.4 常用诀窍 (tricks)

绝大部分诀窍
并非“新技术”

□ 预训练 + 微调

- 预训练: 监督逐层训练，每次训练一层隐结点
- 微调：预训练全部完成后, 对全网络进行微调训练，通常使用 BP算法

可视为将大量参数分组，对每组先找到较好的局部配置，再全局寻优

□ 权共享 (weight-sharing)

减少需优化的参数

- 一组神经元使用相同的连接权值

□ Dropout

可能：降低 Rademacher 复杂度

- 在每轮训练时随机选择一些隐结点令其权重不被更新(下一轮可能被更新)

□ ReLU (Rectified Linear Units)

求导容易；可能：缓解梯度消失现象

- 将 Sigmoid 激活函数修改为修正线性函数 $f(x) = \max(0, x)$

深度学习最重要的特征： 表示学习 、联合优化

传统做法：

Feature Engineering

人工设计特征 ⇨ 学习分类

深度学习：

Representation learning

学习特征 ⇔ 学习分类

所谓 end-to-end Learning （并非新东西 ）

大数据、
高性能计算设备

深度学习常用软件包

- CAFFE (Berkeley Vision and Learning Center, BVLC)

  http://caffe.berkeleyvision.org/

- MatConvNet (Oxford Visual Geometry Group, VGG)

  http://www.vlfeat.org/matconvnet/

- Torch

  http://torch.ch/

- … …